P^*_n , may not be a complete block.

For each mode, the encryption process transforms every plaintext data block or segment into a corresponding ciphertext data block or segment with the same bit length, so that the ciphertext is a sequence of data blocks or segments. The ciphertext is denoted as follows: for the ECB and CBC modes, $C_1, C_2,\ldots, C_n$; for the CFB mode, $C^{\#}_1, C^{\#}_2,\ldots, C^{\#}_n$; and, for the OFB and CTR modes, $C_1, C_2,\ldots, C_{n-1}, C^*_n$, where C^*_n may be a partial block.

The formatting of the plaintext, including in some cases the appending of padding bits to form complete data blocks or data segments, is outside the scope of this recommendation. Padding is discussed in Appendix A.

5.3 Initialization Vectors

The input to the encryption processes of the CBC, CFB, and OFB modes includes, in addition to the plaintext, a data block called the initialization vector (IV), denoted *IV*. The IV is used in an initial step in the encryption of a message and in the corresponding decryption of the message.

The IV need not be secret; however, for the CBC and CFB modes, the IV for any particular execution of the encryption process must be unpredictable, and, for the OFB mode, unique IVs must be used for each execution of the encryption process. The generation of IVs is discussed in Appendix C.

5.4 Examples of Operations and Functions

The concatenation operation on bit strings is denoted | ; for example, 001 | 10111 = 00110111.

Given bit strings of equal length, the exclusive-OR operation, denoted $\oplus$, specifies the addition, modulo 2, of the bits in each bit position, i.e., without carries. Thus, 10011 $\oplus$ 10101= 00110, for example.

The functions LSB_s and MSB_s return the *s* least significant bits and the *s* most significant bits of their arguments. For example, $LSB_3(111011010) = 010$, and $MSB_4(111011010) = 1110$.

Given a positive integer *m* and a non-negative (decimal) integer *x* that is less than 2^m, the binary representation of *x* in *m* bits is denoted $[x]_m$. For example, $[45]_8 = 00101101$.

6 Block Cipher Modes of Operation

The mathematical specifications of the five modes are given in Sections 6.1-6.5, along with descriptions, illustrations, and comments on the potential for parallel processing.

6.1 The Electronic Codebook Mode

The Electronic Codebook (ECB) mode is a confidentiality mode that features, for a given key, the assignment of a fixed ciphertext block to each plaintext block, analogous to the assignment of code words in a codebook. The Electronic Codebook (ECB) mode is defined as follows:

ECB Encryption: $C_j = CIPH_K(P_j)$ for $j = 1 \ldots n$.

ECB Decryption: $P_j = CIPH^{-1}_K(C_j)$ for $j = 1 \ldots n$.

In ECB encryption, the forward cipher function is applied directly and independently to each block of the plaintext. The resulting sequence of output blocks is the ciphertext.

In ECB decryption, the inverse cipher function is applied directly and independently to each block of the ciphertext. The resulting sequence of output blocks is the plaintext.

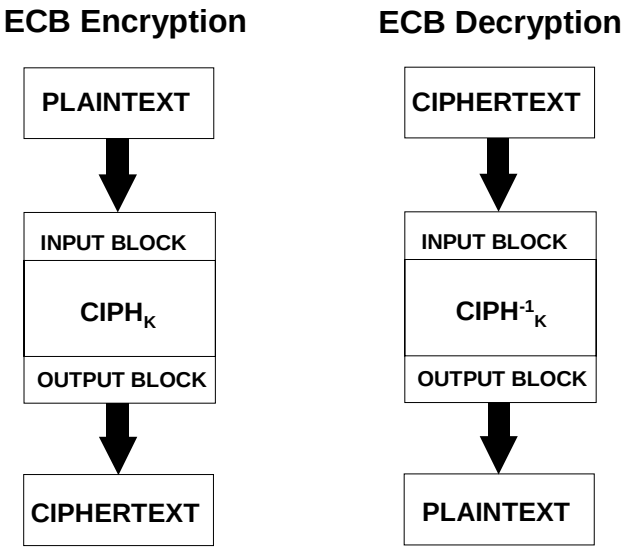

Figure 1: The ECB Mode

In ECB encryption and ECB decryption, multiple forward cipher functions and inverse cipher functions can be computed in parallel.

In the ECB mode, under a given key, any given plaintext block always gets encrypted to the

using the same IV to be easily recovered from the *j*th ciphertext block of that message.

Confidentiality may similarly be compromised if *any* of the input blocks to the forward cipher function for the encryption of a message is designated as the IV for the encryption of another message under the given key.

The OFB mode is illustrated in Figure 4.

6.5 The Counter Mode

The Counter (CTR) mode is a confidentiality mode that features the application of the forward cipher to a set of input blocks, called counters, to produce a sequence of output blocks that are exclusive-ORed with the plaintext to produce the ciphertext, and vice versa. The sequence of counters must have the property that each block in the sequence is different from every other block. This condition is not restricted to a single message: across all of the messages that are encrypted under the given key, all of the counters must be distinct. In this recommendation, the counters for a given message are denoted $T_1, T_2, \ldots, T_n$. Methods for generating counters are discussed in Appendix B. Given a sequence of counters, $T_1, T_2, \ldots, T_n$, the CTR mode is defined as follows:

CTR Encryption:	$O_j = CIPH_K(T_j)$	for $j = 1, 2 \ldots n$;
	$C_j = P_j \oplus O_j$	for $j = 1, 2 \ldots n\text{-}1$;
	$C^*_n = P^*_n \oplus MSB_u(O_n)$.	
CTR Decryption:	$O_j = CIPH_K(T_j)$	for $j = 1, 2 \ldots n$;
	$P_j = C_j \oplus O_j$	for $j = 1, 2 \ldots n\text{-}1$;
	$P^*_n = C^*_n \oplus MSB_u(O_n)$.	

In CTR encryption, the forward cipher function is invoked on each counter block, and the resulting output blocks are exclusive-ORed with the corresponding plaintext blocks to produce the ciphertext blocks. For the last block, which may be a partial block of *u* bits, the most significant *u* bits of the last output block are used for the exclusive-OR operation; the remaining *b-u* bits of the last output block are discarded.

In CTR decryption, the forward cipher function is invoked on each counter block, and the resulting output blocks are exclusive-ORed with the corresponding ciphertext blocks to recover the plaintext blocks. For the last block, which may be a partial block of *u* bits, the most significant *u* bits of the last output block are used for the exclusive-OR operation; the remaining *b-u* bits of the last output block are discarded.

In both CTR encryption and CTR decryption, the forward cipher functions can be performed in parallel; similarly, the plaintext block that corresponds to any particular ciphertext block can be recovered independently from the other plaintext blocks if the corresponding counter block can be determined. Moreover, the forward cipher functions can be applied to the counters prior to the availability of the plaintext or ciphertext data.

Appendix B: Generation of Counter Blocks

The specification of the CTR mode requires a unique counter block for each plaintext block that is ever encrypted under a given key, across all messages. If, contrary to this requirement, a counter block is used repeatedly, then the confidentiality of all of the plaintext blocks corresponding to that counter block may be compromised. In particular, if any plaintext block that is encrypted using a given counter block is known, then the output of the forward cipher function can be determined easily from the associated ciphertext block. This output allows any other plaintext blocks that are encrypted using the same counter block to be easily recovered from their associated ciphertext blocks.

There are two aspects to satisfying the uniqueness requirement. First, an incrementing function for generating the counter blocks from any initial counter block can ensure that counter blocks do not repeat within a given message. Second, the initial counter blocks, T_1, must be chosen to ensure that counters are unique across all messages that are encrypted under the given key.

B.1 The Standard Incrementing Function

In general, given the initial counter block for a message, the successive counter blocks are derived by applying an incrementing function. As in the above specifications of the modes, n is the number of blocks in the given plaintext message, and b is the number of bits in the block.

The standard incrementing function can apply either to an entire block or to a part of a block. Let m be the number of bits in the specific part of the block to be incremented; thus, m is a positive integer such that $m \leq b$. Any string of m bits can be regarded as the binary representation of a non-negative integer x that is strictly less than 2^m. The standard incrementing function takes $[x]_m$ and returns $[x+1 \bmod 2^m]_m$.

For example, let the standard incrementing function apply to the five least significant bits of eight bit blocks, so that b=8 and m=5 (unrealistically small values); let * represent each unknown bit in this example, and let ***11110 represent a block to be incremented. The following sequence of blocks results from four applications of the standard incrementing function:

* * * 1 1 1 1 0
* * * 1 1 1 1 1
* * * 0 0 0 0 0
* * * 0 0 0 0 1
* * * 0 0 0 1 0.

Counter blocks in which a given set of m bits are incremented by the standard incrementing function satisfy the uniqueness requirement *within the given message* provided that $n \leq 2^m$. Whether the uniqueness requirement for counter blocks is satisfied across all messages that are encrypted under a given key then depends on the choices of the initial counter blocks for the messages, as discussed in the next section.

Appendix D: Error Properties

A bit error is the substitution of a '0' bit for a '1' bit, or vice versa. This appendix contains a discussion of the effects of bit errors in ciphertext blocks (or segments), counter blocks, and IVs on the modes in this recommendation. Insertion or deletion of bits into ciphertext blocks (or segments) is also discussed.

For any confidentiality mode, if there are any bit errors in a single ciphertext block (or segment), then the decryption of that ciphertext block (or segment) will be incorrect, i.e., it will differ from the original plaintext block (or segment). In the CFB, OFB, and CTR modes, the bit error(s) in the decrypted ciphertext block (or segment) occur in the same bit position(s) as in the ciphertext block (or segment); the other bit positions are not affected. In the ECB and CBC modes, a bit error may occur, independently, in any bit position of the decrypted ciphertext block, with an expected error rate of fifty percent, depending on the strength of the underlying block cipher.

For the ECB, OFB, and CTR modes, bit errors within a ciphertext block do not affect the decryption of any other blocks. In the CBC mode, any bit positions that contain bit errors in a ciphertext block will also contain bit errors in the decryption of the succeeding ciphertext block; the other bit positions are not affected. In the CFB mode, bit errors in a ciphertext segment affect the decryption of the next b/s (rounded up to the nearest integer) ciphertext segments. A bit error may occur, independently, in any bit position in these decrypted segments, with an expected error rate of fifty percent.

Similarly, for the CTR mode, if there is a bit error in a counter block, then a bit error may occur, independently, in any bit position of the decryption of the corresponding ciphertext, with an expected error rate of fifty percent.

Bit errors in IVs also affect the decryption process. In the OFB mode, bit errors in the IV affect the decryption of every ciphertext block. In the CFB mode, bit errors in the IV affect, at a minimum, the decryption of the first ciphertext segment, and possibly successive ciphertext segments, depending on the bit position of the rightmost bit error in the IV. (In general, a bit error in the ith most significant bit position affects the decryptions of the first i/s (rounding up) ciphertext segments.) For both the OFB and CFB modes, a bit error may occur, independently, in any bit position of the affected ciphertext blocks (or segments), with an expected error rate of fifty percent. In the CBC mode, if bit errors occur in the IV, then the first ciphertext block will be decrypted incorrectly, and bit errors will occur in exactly the same bit positions as in the IV; the decryptions of the other ciphertext blocks are not affected.

Consequently, for the CBC mode, the decryption of the first ciphertext block is vulnerable to the (deliberate) introduction of bit errors in specific bit positions of the IV if the integrity of the IV is not protected. Similarly, for the OFB and CTR modes, the decryption of any ciphertext block is vulnerable to the introduction of specific bit errors into that ciphertext block if its integrity is not protected. The same property also holds for the ciphertext segments in the CFB mode; however, for every ciphertext segment except the last one, the existence of such bit errors may be detected by their randomizing effect on the decryption of the succeeding ciphertext segment.